

Linköpings Universitet

Global Illumination & Rendering

TNCG15

Monte Carlo Raytracer

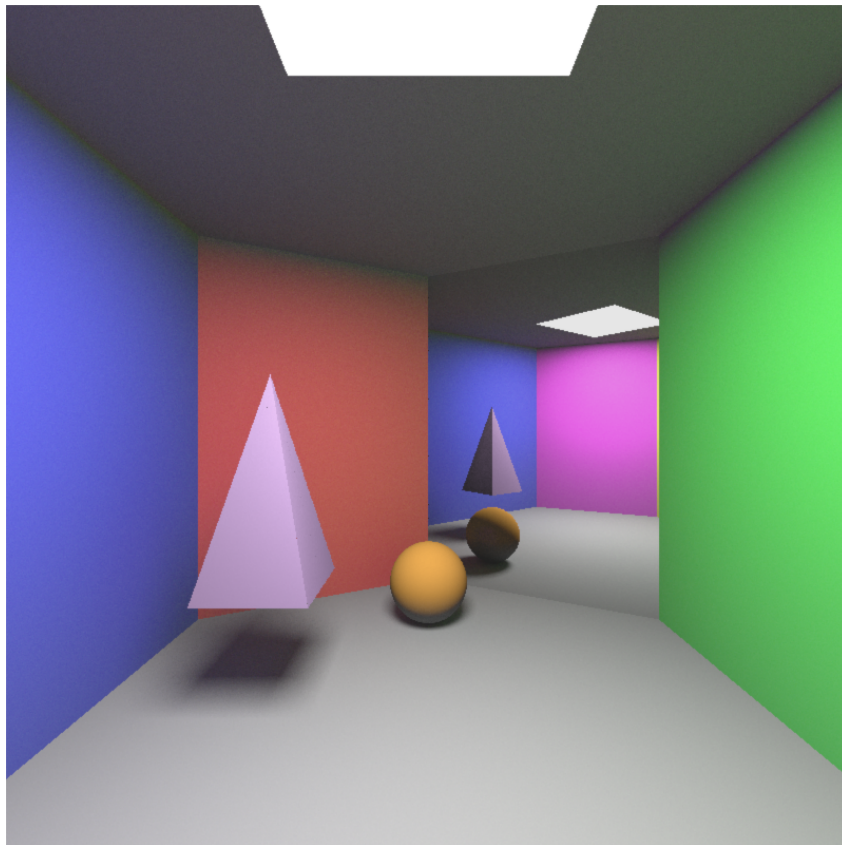
STUDENT:

Joel Hellberg - joehe737

Linus Palm - linpa097

EXAMINATOR:

Mark E. Dieckmann



November 20, 2025

Abstract

This report presents the development and implementation of a Monte Carlo raytracer designed to simulate global illumination in simple 3D scenes. The project focuses on rendering environments composed of rectangles, triangles, and spheres with Lambertian and reflective materials.

Rays are traced from the camera through the scene, interacting with surfaces to capture both direct and indirect lighting, including shadows and reflections. The report details the mathematical foundations of ray-surface intersections, light, camera and the different materials.

Additionally, the software architecture is described, including classes for shapes, materials, lights, rays, and the camera. Results demonstrate the effectiveness of Monte Carlo path tracing in producing realistic images, highlighting the impact of sampling, image resolution, and Russian roulette termination on rendering quality and performance.

The implemented approach demonstrates how Monte Carlo ray tracing can be used to produce semi-realistic images and served as an exercise to provide the students with deeper knowledge and understanding of global illumination and raytracing.

CONTENTS

1	Introduction	3
1.1	Introduction to the Field	4
1.2	Purpose	4
1.3	Limitations	5
2	Background	6
2.1	3D Environment Setup	6
2.1.1	3D Environment Setup: Rectangles	6
2.1.2	3D Environment Setup: Triangles	7
2.1.3	3D Environment Setup: Spheres	7
2.2	Camera Setup	7
2.3	Rays	7
2.4	Ray-Surface Intersections	8
2.4.1	Ray-Surface Intersections: Rectangles	8
2.4.2	Ray-Surface Intersections: Triangles	9
2.4.3	Ray-Surface Intersections: Spheres	10
2.5	Mirror - Perfect Reflector	11
2.6	Light source	12
2.7	Lambertian Surfaces	12
2.7.1	Reflections	12
2.7.2	Russian Roulette	13

3	Method	14
3.1	Software	14
3.2	Implementation	14
3.2.1	ColorDBL	15
3.2.2	Material	15
3.2.3	Shape	15
3.2.4	Sphere	15
3.2.5	Light	16
3.2.6	Ray	16
3.2.7	Camera	16
3.2.8	Scene	16
3.3	Class diagram	16
4	Results	18
4.1	Environment	18
4.2	Rendering Variables	19
4.3	Final Image	21
5	Discussion and Conclusion	22

1. INTRODUCTION

Since the start of computer graphics there has always been a desire to make movies, games and other applicable fields and environments look more realistic. Over the past decades movies and especially games have evolved from being unrealistic games with few details, simple objects, structures and materials to scenes where it's hard to determine whether it's real or computer generated. This due to their complex structures, detailed objects and materials, and lifelike lighting that make them appear almost real.

Multiple game franchises has been around for a long time and it's therefore easy to compare the big differences and detect the large impact of computer graphics. The Call of duty[2] is one of those franchises and Figure 1.1 and Figure 1.2 show two different games, the first and the latest in their franchise. The first Call of duty was released in 2003 and the latest were released in 2024. Only 21 years separates their release but their graphical difference highlights how far computer graphics have advanced in a very short amount of time.



Figure 1.1: In-game screenshot[1] of the first game in the Call of Duty-franchise called Call of Duty released in 2003.



Figure 1.2: In-game screenshot[5] of the latest game in the Call of Duty-franchise called Call of Duty: Black Ops 6 released in 2024.

1.1 Introduction to the Field

Global illumination models in 3D computer graphics include different methods to simulate light and how it interacts with different materials and surfaces in a scene to create a more realistic environment. To achieve this most methods include both direct and indirect light. Direct light is light that interact with a surface directly from the lightsource and indirect light is reflected light to create soft shadows. These methods are expensive to compute but later and more powerful components for computers enable global illumination to be used in more applicable fields.

The method used and described in this report is a Monte Carlo raytracer. The concept for a Monte Carlo raytracer is that for every pixel on the image plane we emit several rays. The color or intensity of the pixel is determined based on which surfaces the rays intersect and how light interacts with those surfaces, including effects such as reflection, refraction, and shadows. By averaging the contributions of many rays per pixel, Monte Carlo ray tracing reduces noise and approximates global illumination, capturing indirect lighting, caustics, and soft shadows. This method allows for more physically realistic rendering compared to traditional ray tracing, especially in scenes with complex light properties[6].

1.2 Purpose

The purpose of the project was to create a raytracer to render an environment with different geometric shapes and materials to simulate global illumination. The project were performed by 2 students at Linköping University during the course Advanced Global Illumination and Rendering taught by Mark E. Dieckmann.

The purpose of the report is to explain the used theory behind the Monte Carlo raytracer and how the implementations were done in the project together with resulting images.

1.3 Limitations

Due to the tight timeframe for the project during the course the environment is limited to simple shapes with only specular and diffuse materials.

Due to the advanced and different methods that is used to create global illumination the report will only cover the theory used to create a simple Monte Carlo raytracer. The report will therefore only cover the theory and techniques used in the project and it's structure but not the implementation itself.

The report assumes that the reader has studied linear algebra or has equivalent knowledge in this area. It will therefore not explain basic concepts such as vectors, normals, dot products and cross products.

2. BACKGROUND

The theory discussed in this chapter is based on the lecture slides provided by Mark E. Dieckmann for the course Advanced Global Illumination and Rendering[3].

2.1 3D Environment Setup

To create a three-dimensional environment you need to define the scene by defining several components. The key components is geometry objects, materials, light sources, and the camera. In the following section the methods and equations used for defining these components will be discussed along with how rays intersect with these surfaces within the scene.

2.1.1 3D Environment Setup: Rectangles

The rectangles in the 3D environment are defined by 4 vertices. Each vertex is defined by a x, y and z-coordinate and a material. The normal of each rectangle must have the direction towards the center of the scene, therefore the vertices must be set in a specific order to follow the right-hand rule.

To calculate the normal, two vectors of the rectangle are calculated by

$$D_1 = v_1 - v_0 \quad (2.1)$$

$$D_2 = v_2 - v_1 \quad (2.2)$$

where v_0 , v_1 , v_2 and v_3 represent the vertices of the rectangle and D_1 and D_2 are two orthogonal edges on the rectangle, see Figure 2.1. The normal of the rectangle is then calculated by

$$N = D_1 \times D_2 \quad (2.3)$$

where N represent the normal.

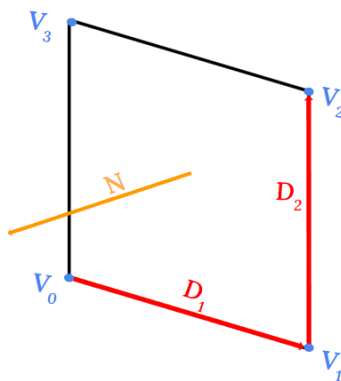


Figure 2.1: Visualization of a rectangle with vertices, normal and vectors.

2.1.2 3D Environment Setup: Triangles

As rectangles, the triangles are defined by vertices with x,y and z-coordinates. For triangles to work with future algorithms they need to be right-angled triangles and a triangle is therefore often split into two. The normal is then calculated the same way as described for rectangles, see equation 2.1, 2.2 and 2.3.

2.1.3 3D Environment Setup: Spheres

The sphere is defined by a radius and its center, defined by x,y and z-coordinates.

2.2 Camera Setup

To be able to render a two dimensional image of a three dimensional environment a camera is needed to act as viewpoint. The camera can capture the desired area of the scene by adjusting the cameras position set by x, y and z-coordinates.

Infront of the camera is a camera plane with a set grid of pixels, depending on the desired size. The pixels will be influenced by the rays, casted from the camera, through the camera plane into the scene until they hit a surface. It's the camera plane with it's pixels that will be rendered into the image representing the environment.

2.3 Rays

To be able to define a three dimensional scene a tool to go through the scene and interact with all objects is needed. The tool used for this is a ray that will travel from the camera

into the scene to create the two dimensional visualization of the three dimensional environment.

Described as a cylinder with an infinitely small diameter rays are used differently depending on it's purpose. Primary rays are sent trough all pixels in the camera plane to interact with the scene to see what the specific pixel should be influenced by. Lightrays measures how much light a point is influenced by relative to the position of the light-source. Shadowrays are used to check wether a point in the scene is hidden by another object or not and therefore should or should not be influenced by the lightsource.

2.4 Ray-Surface Intersections

When rays are sent from the camera they can interract with the scene. As described in section 2.1 the normal direction needs to point towards the center of the scene. If a ray intersect with a surface or not are based on the ray-direction and the normal of the surface. If the dotproduct between these vectors are below zero they do not point towards eachother and can therefore not intersect. This criteria are checked at every intersection and a vital part of the method to represent the three dimensional scene on a two dimensional image.

To determine the path for a ray the following equation is used

$$x(t) = p_s + t(p_e - p_s) \quad (2.4)$$

where $x(t)$ represent a specific intersection point for a ray on a surface, p_s represent the starting point for the ray, p_e represent the endpoint. The vector given by $p_e - p_s$ is called the direction vector D for the ray and t is the positive variable that need to be multiplied with D for it to be able to reach a surface. Depending on which type of surface the ray can hit different methods is used to obtain the value t described in the following section.

2.4.1 Ray-Surface Intersections: Rectangles

The intersection between a ray and a rectangular surface can be calculated by the ray-equation with the specific t -value. The t -values are calculated by

$$t = \frac{(v - s) \cdot N}{N \cdot d} \quad (2.5)$$

where v represent a vertex, s the starting point for the ray, d the direction of the ray and N the normal of the surface. Thus this t -value could represent an intersection point if the rectangle were to be an endless plane. To calculate if the intersection point are inside the specified rectangle we need to calculate two variables, a and b . a and b are calculated by

$$a = \frac{(x_i - v) \cdot c_1}{c_1 \cdot c_1} \quad (2.6)$$

$$b = \frac{(x_i - v) \cdot c_2}{c_2 \cdot c_2} \quad (2.7)$$

where x_i represent the potential intersection point calculated by the ray-equation, v the vertex and c_1 and c_2 are two orthogonal vectors on the rectangles edge with their starting point in v and pointing away from it.

If

$$0 \leq a \leq 1 \quad (2.8)$$

$$0 \leq b \leq 1 \quad (2.9)$$

the intersection point are inside the specified rectangle.

2.4.2 Ray-Surface Intersections: Triangles

For triangles the same method used for rectangle can be used but the Möller Trumbore algorithm is a faster way to compute a intersection point on a triangular surface. The algorithm use barycentric coordinates (u, v)

$$T(u, v) = (1 - u - v) \cdot v_0 + u \cdot v_1 + v \cdot v_2, \quad u, v \geq 0 \quad \text{and} \quad u + v \leq 1 \quad (2.10)$$

where $T(u, v)$ describes a point on the triangles surface and v_0, v_1 and v_2 are the vertices of the triangle. By combining 2.10 and the ray-equation 2.4 the following equation is obtained

$$(1 - u - v)\mathbf{v}_0 + u\mathbf{v}_1 + v\mathbf{v}_2 = \mathbf{p}_s + t(\mathbf{p}_e - \mathbf{p}_s) \quad (2.11)$$

and then perform the following variable substitution

$$\begin{aligned} T &= p_s - v_0, \\ E_1 &= v_1 - v_0, \\ E_2 &= v_2 - v_0, \\ D &= p_e - p_s, \\ P &= D \times E_2, \\ Q &= T \times E_1. \end{aligned} \quad (2.12)$$

If the variables in 2.12 are simplified we obtain the following equation for t

$$t = \frac{Q \cdot E_2}{P \cdot E_1} \quad (2.13)$$

and the t -value for the ray-equation can be calculated. The intersection point on the triangular surface is then given by inserting the given t -value from equation 2.13 in the ray equation 2.4.

2.4.3 Ray-Surface Intersections: Spheres

Since spheres doesn't have a defined normal but a normal on every point on it's surface the combination between the ray-equation and the equation for a sphere

$$(X_c - C)^2 = r^2 \quad (2.14)$$

where X_c represent a surface point on the sphere, C the spheres center and r the radius. The composition of the equations

$$r^2 = (S - C) \cdot (S - C) + 2D \cdot (S - C)t + D \cdot Dt^2 \quad (2.15)$$

and with the following variable substitution

$$c_1 = D \cdot D \quad (2.16)$$

$$c_2 = 2D \cdot (S - C) \quad (2.17)$$

$$c_3 = (S - C) \cdot (S - C) - r^2 \quad (2.18)$$

the following equation is given

$$c_1 t^2 + c_2 t + c_3 = 0 \Leftrightarrow t = \frac{-c_2 \pm \sqrt{c_2^2 - 4c_1 c_3}}{2c_1} \quad (2.19)$$

Since the equation for t include a squareroot and there's a possibility the variables inside the squareroot equals a negative number. Since t must be a real number the following argument

$$\arg = c_2^2 - 4c_1 c_3 \quad (2.20)$$

is set. The argument can tell if the ray hit the sphere or not by the following statements:

$$\textbf{If } \arg < 0 : \text{No intersection point} \quad (2.21)$$

$$\textbf{If } \arg = 0 : \text{One intersection point} \quad (2.22)$$

$$\textbf{If } \arg > 0 : \text{Two intersection points} \quad (2.23)$$

The third statement for $\arg$ (see equation 2.23) give two different intersection points on the sphere surface. The visible intersection point on the sphere from the camera is the closest one, therefore the smallest number for t , $t_{\min}$ will be used as an input in the ray equation, see Figure 2.2.

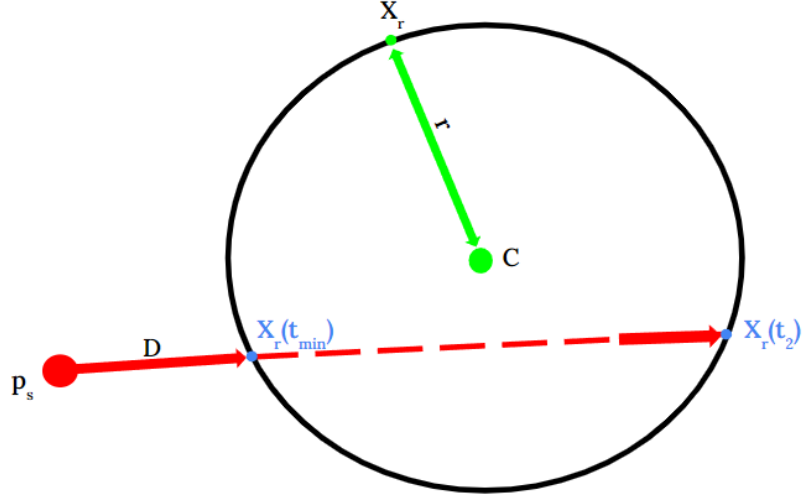


Figure 2.2: Visualization of intersection with sphere with two different values for t .

When the ray hits a spheric surface the need is still to check if it's visible for the viewer by looking at the direction of the normal. As explained the normal can only be calculated for a specific surface point with the equation

$$\vec{N} = \frac{x(t_{\min}) - C}{\|x(t_{\min}) - C\|} \quad (2.24)$$

where $x(t_{\min})$ represent the closest intersection point and C the coordinates of the center of the sphere.

2.5 Mirror - Perfect Reflector

Implementation and methods for different surfaces are needed in the environment but also for different materials. A mirror, in this case a perfect reflector reflect incoming light without any absorption or scatter the outgoing light. This causes the outgoing light to have the same angle relative to the mirror as the incoming light and will also reflect 100% of its intensity, see Figure 2.3.

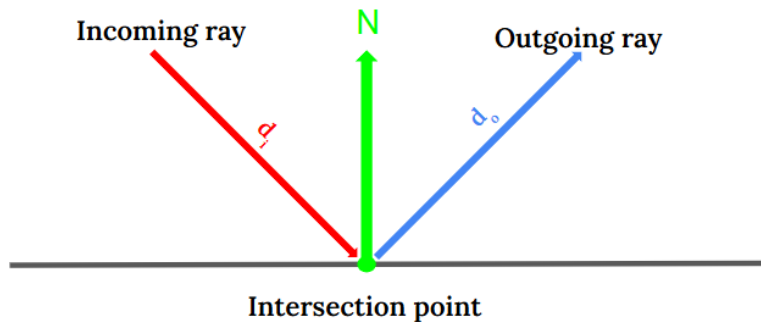


Figure 2.3: Visualization of reflection on a mirror.

The direction of the reflected ray is calculated by

$$\vec{d}_o = \vec{d}_i - 2(\vec{d}_i \cdot \vec{N})\vec{N} \quad (2.25)$$

where $\vec{d}_o$ is the resulting reflected ray, $\vec{d}_i$ the incoming ray and $\vec{N}$ the surface normal.

The properties, like the importance value of the incoming ray is copied to the reflected ray to achieve the perfect reflector.

2.6 Light source

A light source in a 3D scene represents an area or point that emits light and illuminates objects in the environment. In this project, the light source is defined as an area light.

To evaluate the illumination contributed by an area light source, multiple sample positions are selected across the light's surface, typically using uniform or stratified sampling strategies. For each sampled position, a shadow ray is cast from the shading point toward the light. If this ray reaches the sampled position without intersecting any scene geometry, the sample is considered unoccluded and contributes to the direct lighting; otherwise, it is discarded as being in shadow.

For each unoccluded sample, the contribution to the irradiance is computed according to equation:

$$\frac{\cos(X) \cdot \cos(Y)}{d^2} \quad (2.26)$$

In this expression, X denotes the angle between the direction to the light sample and the surface normal at the shading point, Y denotes the angle between the light's surface normal and the direction toward the shading point, d is the distance between the shading point and the sampled position on the light's surface.

Each sample's contribution accounts for geometric factors such as the cosine terms associated with surface orientation and light emission, as well as distance attenuation. The total irradiance from the area light is then approximated by averaging the contributions from all unoccluded samples.

2.7 Lambertian Surfaces

2.7.1 Reflections

Diffuse (or Lambertian) reflection describes the phenomenon in which incoming light is scattered broadly across many directions rather than being reflected along a single mirror

direction. For an ideal Lambertian surface, the outgoing radiance is independent of the viewer’s direction. Mathematically, this is captured by a BRDF of the form

$$f_r = \frac{\rho}{\pi}, \quad (2.27)$$

where ρ is the surface albedo, and the $\frac{1}{\pi}$ term ensures proper energy conservation under hemispherical scattering.

The observed irradiance also follows Lambert’s cosine law, scaling with the cosine of the angle between the incident light direction and the surface normal.

$$\cos(\theta) = \frac{n \cdot w}{|n| |w|} \quad (2.28)$$

In a ray-tracing context, diffuse reflection is naturally interpreted through Monte Carlo sampling. When a ray strikes a diffuse surface, it does not reflect in a single deterministic direction (as in specular reflection). Instead, a new direction is sampled randomly from the hemisphere oriented around the surface normal.

In ray tracing, diffuse reflection is modeled using Monte Carlo sampling. When a ray hits a diffuse surface, it is not reflected in a single direction as in specular materials. Instead, a new direction is chosen randomly from the hemisphere around the surface normal.

2.7.2 Russian Roulette

To compute the radiance coming from many incoming directions on a Lambertian surface, Monte Carlo path tracing often uses a technique called Russian roulette. Rather than allowing every ray to bounce indefinitely, at each bounce we generate a random number to decide whether the path should terminate. This reduces unnecessary computations for rays that would contribute very little to the final pixel-value while still keeping the estimator unbiased.

If the ray survives the Russian roulette check, it is scattered in a random direction over the hemisphere. Although a single camera ray only produces one such path, we perform many samples per pixel and average the results. This sampling from many directions allows us to approximate global illumination and capture the diffuse reflections that occur in the real world.

3. METHOD

This chapter describes the implementation of the project with its different classes for shapes, materials, light sources, rays, the camera, and the scene. It also describes where the equations described in chapter 2 were utilized in the project.

3.1 Software

The project was conducted using Visual Studio 2022 for the implementations of the described methods and environments in C++, using OpenGL and the GLM library for mathematical operations such as vectors, dot products, cross products. The same IDE was also used to render the images of the scene.

For visualization in an early stage without a visible scene on Visual Studio GeoGebra was used. GeoGebra gave a visual understanding of the scene and the surface normals, see Figure 4.2 before any rendered image could be obtained. In a later stage of the project when simple images of the environment could be rendered using the methods discussed in section 2 the image editor GIMP was used to convert these rendered ppm-files into visible images.

The project were performed by 2 students, therefore the Git-client Fork was used to manage a Git repository and easier implementation across separate devices.

3.2 Implementation

For implementation and a clear code structure several classes were created. The project was divided into separate components to handle different parts such as geometry, materials, light, and rays. This approach improved readability, structure and made debugging easier.

3.2.1 ColorDBL

The ColorDBL class stores RGB color values as doubles and includes functions for color manipulation, clamping, and conversion to vectors. It provides the foundation for all color handling, including material colors and light intensity.

3.2.2 Material

Material defines the visual and physical properties of each surface. It contains a ColorDBL instance and what type of material a surface had such as reflective, lambertian, or light-emitting. The class allows easy adjustment of color and brightness for different object types.

3.2.3 Shape

The baseclass Shape consisted of functions common for the polygons, rectangles and triangles. It defined shared attributes and methods for computing normals, intersections, and retrieving color and material data.

Rectangle

Rectangle is a subclass of Shape that defines a surface with four vertex points. It includes specific intersection and normal calculations used for ray interactions with rectangular surfaces. The class utilized the equations explained in section 2.4.1.

Triangle

The Triangle class extends Shape in a similar manner as Rectangle, but defines its geometry using three vertex points instead of four. The class utilized the equations explained in section 2.4.2.

3.2.4 Sphere

Sphere represents the spherical object in the scene. Unlike Rectangle and Triangle, it is not derived from Shape since its calculations for intersections and surface normals differs. The class utilized the equations explained in section 2.4.3.

3.2.5 Light

The Light class represents an area light source defined as a rectangle. It contains functions to calculate random points on its surface for sampling purposes, as well as methods for geometric operations like vector edges.

3.2.6 Ray

Ray handles the ray tracing logic in the rendering process. It contains methods for creating reflections and lambertian rays, managing ray paths, and computing light intensity based on intersections and shadows.

3.2.7 Camera

Camera is responsible for generating rays corresponding to each pixel in the image plane. It defines the image dimensions and computes ray directions based on pixel coordinates.

3.2.8 Scene

The Scene class acts as a container for all objects in the environment, including shapes, lights, spheres, and the camera. It is responsible for initializing objects and controlling how rays interact with them during rendering.

3.3 Class diagram

To give an overview of the structure and relationships between the classes, a UML class diagram was created, see Figure 4.9. The diagram shows each class with its attributes and functions. It also highlights inheritance and also shows how classes are connected, indicating which classes contain or make use of other classes.

The class diagram was generated using MermaidChart[4]. The classes, along with its attributes and functions, were provided to MermaidChart where it arranged and visualized the diagram.

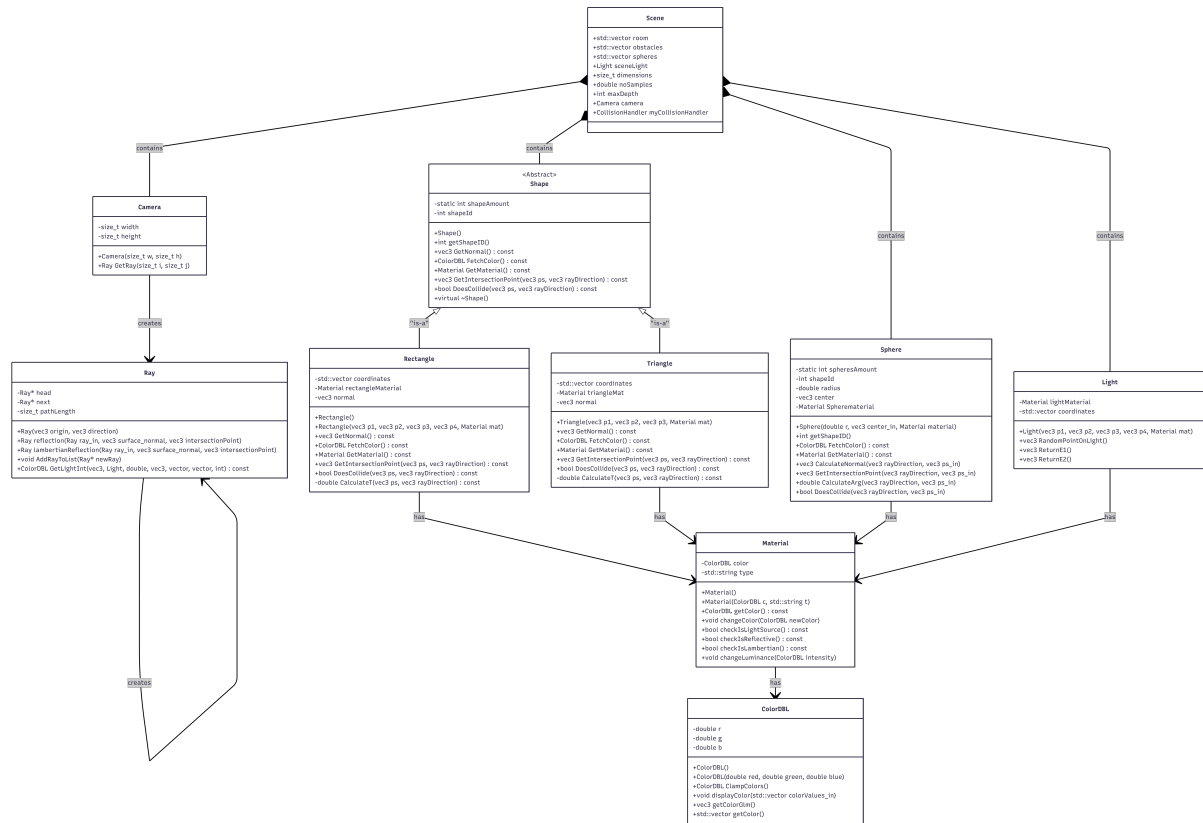


Figure 3.1: UML class diagram showing the classes, attributes, and methods of used in the project.

4. RESULTS

The result of the implemented methods discussed in chapter 2 and chapter 3 will be presented below.

4.1 Environment

The 3D environment with different objects such as rectangles, triangles and spheres as mentioned in chapter 2.1 was the first thing to be defined and implemented. The layout of the scene can be seen in Figure 4.1 with objects, camera and the camera plane. The perfect reflector discussed in section 2.5 is represented by the wall "Mirror", all other objects and walls are diffuse surfaces with their respective color as seen in Figure 4.1.

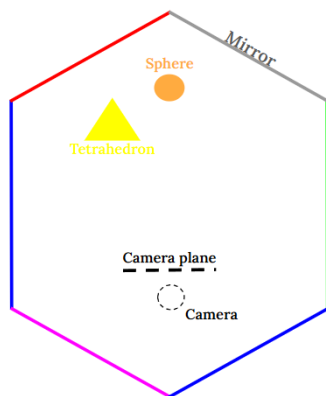


Figure 4.1: 2D top-view of the scene

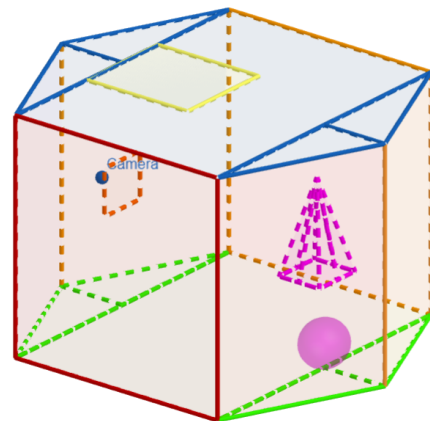


Figure 4.2: Visualization of the scene in GeoGebra

To achieve the rendered images of the 3D environment Geogebra was used to create a visual representation of the scene, see Figure 4.2 before it could be rendered within Visual Studio. The GeoGebra model ensured that the project group could test and verify that all surfaces were correctly defined with respect to their normals.

When the camera setup mentioned in chapter 2.2 and the basic environment discussed above and in chapter 2.1 were implemented together with a lightsource a basic image

could be rendered, see Figure 4.3. In a later stage when the methods for calculating ray-surface intersections mentioned in chapter 2.4 were implemented images from the Monte Carlo ray-tracer could be rendered that gave fairly good results, see the Figures below.

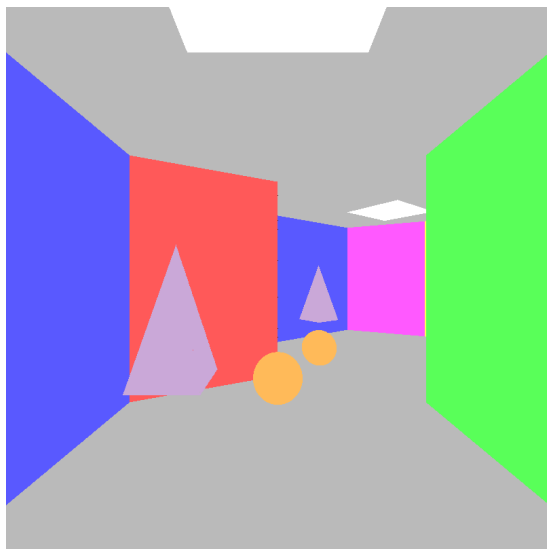


Figure 4.3: Full scene with all components defined.

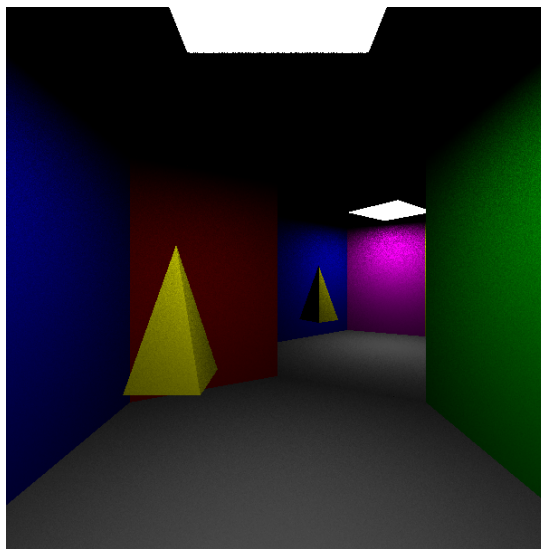


Figure 4.4: Area lighting applied, before shadows.

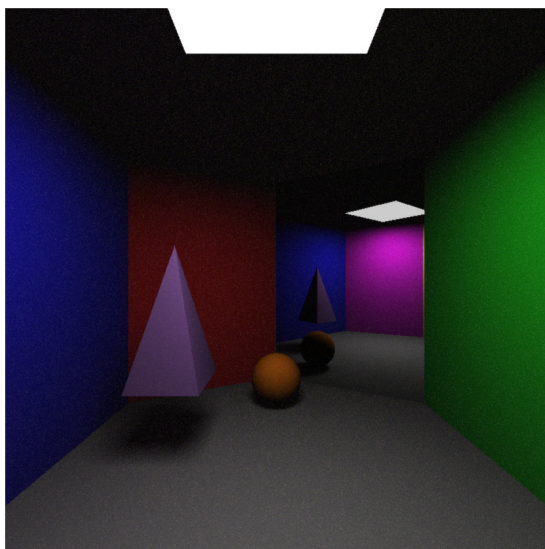


Figure 4.5: Area lighting with shadows.

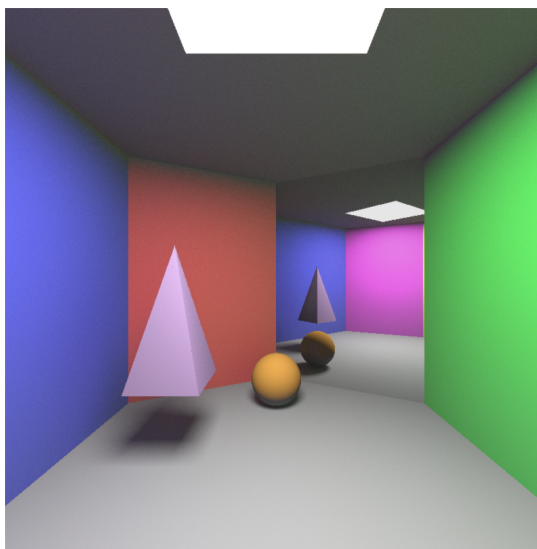


Figure 4.6: Lambertian shading applied.

4.2 Rendering Variables

During the progression of the project several values were tweaked to achieve desired results. The variables are deeply associated with the quality of the image and how long it will take to render.

- dimensions - The size of the images, the output image rendered will be *dimensions* x *dimensions* in size. The higher the value the more detailed image but

leads to longer rendering time.

- noSamples - Number of samples, number of rays traced by each pixel. A higher value leads to a more accurate representation of the environment but longer rendering time.

The two parameters described above both depend on the desired image quality and accuracy. Increasing them improves the visual fidelity of the render but also leads to substantially longer render times. Another parameter affecting both image quality and performance is the Russian roulette termination probability used when a ray strikes a Lambertian surface. A high termination probability reduces the number of bounces, producing faster renders but capturing fewer contributions from surrounding surfaces. A lower probability allows rays to sample more of their environment, improving realism at the cost of longer computation.

In our case, we set the termination probability low and the albedo high in order to emphasize the Lambertian behavior and make its effects more clearly visible in the final images. Changing the image dimensions simply affects the resolution; fewer pixels result in a blurrier render. The number of samples per pixel also has a significant impact: more samples produce smoother, more natural Lambertian color transitions by incorporating a wider range of incoming light directions, while fewer samples lead to noisier, more random-looking pixels, as illustrated in the comparison between figures 4.7 and 4.8.

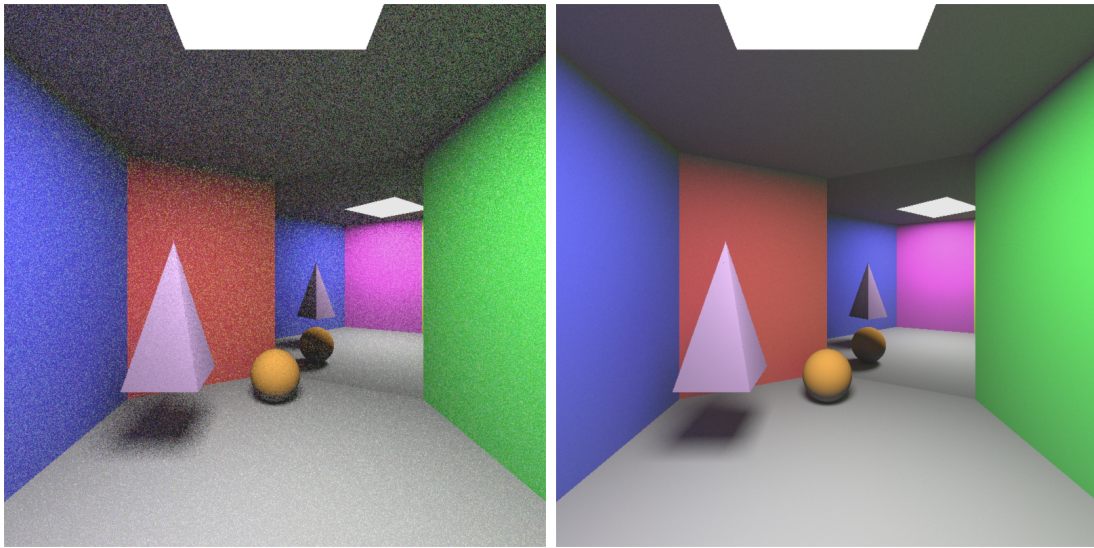


Figure 4.7: Render at 800×800 pixels with 1 sample. Figure 4.8: Render at 800×800 pixels with 100 samples.

4.3 Final Image

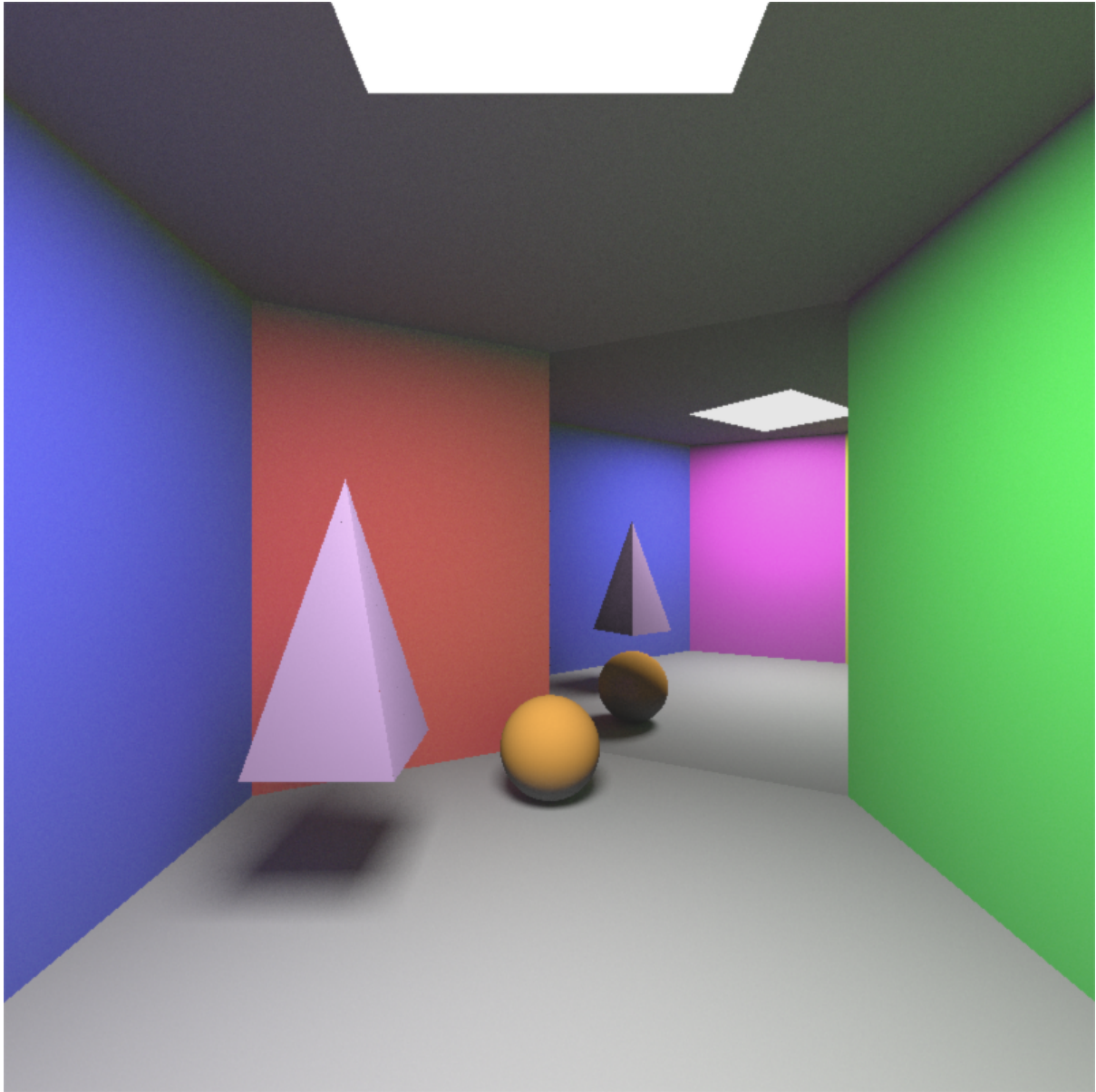


Figure 4.9: Final render at 800×800 pixels with 100 samples, an absorption chance of 0.2, and an albedo of 0.95.

5. DISCUSSION AND CONCLUSION

In summary, our final render incorporates functions and calculations that closely mimic real-world light behavior, resulting in an image of which we are proud and that achieves a convincingly semi-realistic appearance.

We chose a recursive structure for the ray-tracing logic because it provided a cleaner, simpler, and more intuitive implementation while still producing the correct results. However, we are aware of the potential memory implications of deep recursion. Although this approach worked perfectly for our purposes, a more complex renderer with longer light paths would likely require a refactoring toward an iterative structure, first constructing a list of interactions and then evaluating the resulting Lambertian color.

The most challenging part of the implementation was the Lambertian material, as it required more advanced calculations than the other components. Since Lambertian shading involves many stochastic ray bounces, each render became significantly slower. To address this, we introduced multithreading to take advantage of all available CPU cores, which greatly improved performance. Initially, this produced incorrect images, which we mistakenly attributed to mathematical errors. We later discovered the true cause: our multithreaded implementation allowed different threads to share variables that should have been kept thread-local. Once we ensured that each thread used its own independent data and avoided shared state, the render results returned to normal.

REFERENCES

- [1] Aambar. *Call of Duty (2003)*. Accessed: 11 November 2025. 2015. URL: <https://aambar.wordpress.com/2015/11/29/call-of-duty2003/>.
- [2] Call of Duty Wiki – Fandom. *Call of Duty series*. Accessed: 6 November 2025. 2025. URL: https://callofduty.fandom.com/wiki/Call_of_Duty_%28series%29.
- [3] Linköping University. *TNCG15 – Advanced Global Illumination and Rendering*. Accessed: 20 November 2025. 2025. URL: <https://studieinfo.liu.se/kurs/TNCG15>.
- [4] Mermaid. *Mermaid: Generation of Diagrams from Text*. Accessed: 11 November 2025. 2025. URL: <https://mermaid.js.org/>.
- [5] MitchCactus. *Call of Duty: Black Ops 6 Gameplay Walkthrough in 5 Minutes*. Accessed: 5 November 2025. 2024. URL: <https://mitchcactus.co/blog/call-of-duty/call-of-duty-black-ops-6-gameplay-walkthrough-in-5-minutes/>.
- [6] ScienceDirect. *Monte Carlo Ray Tracing*. Accessed: 5 November 2025. 2025. URL: <https://www.sciencedirect.com/topics/chemical-engineering/monte-carlo-ray-tracing>.